

Modern web application development

Web evolution

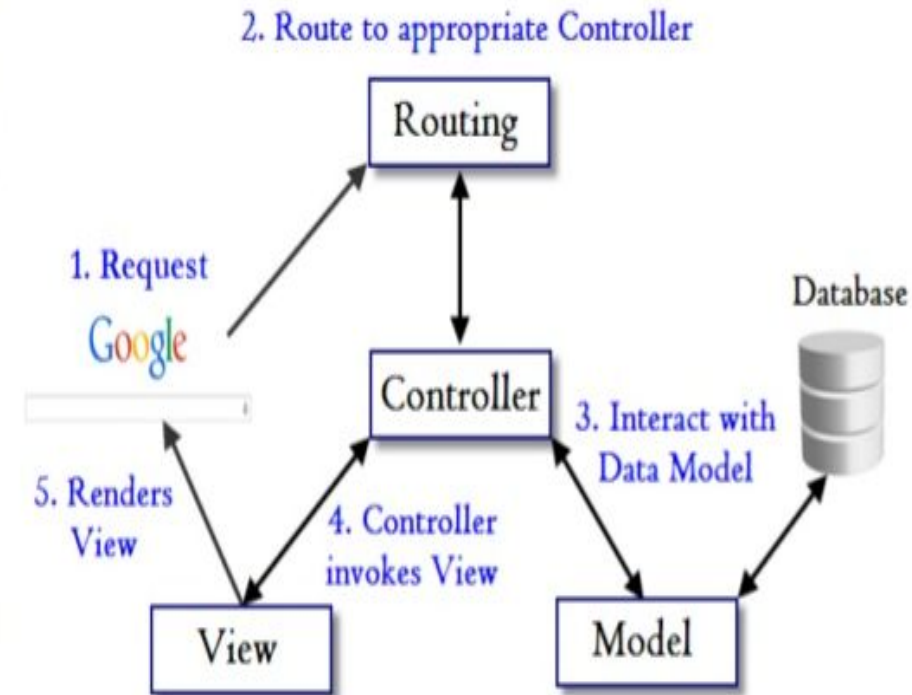
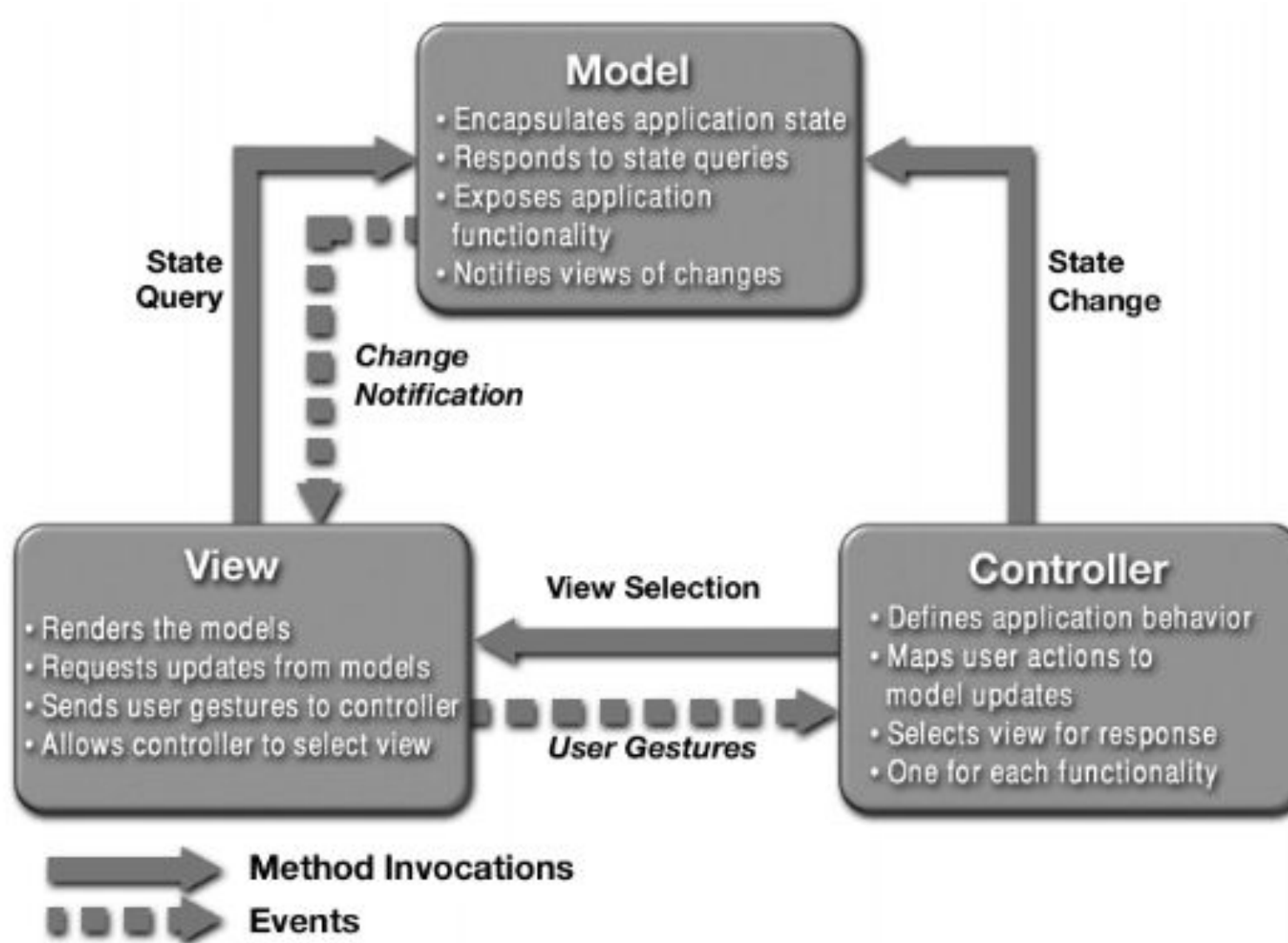
- Web has evolved from simple static pages to **complex real time** pages



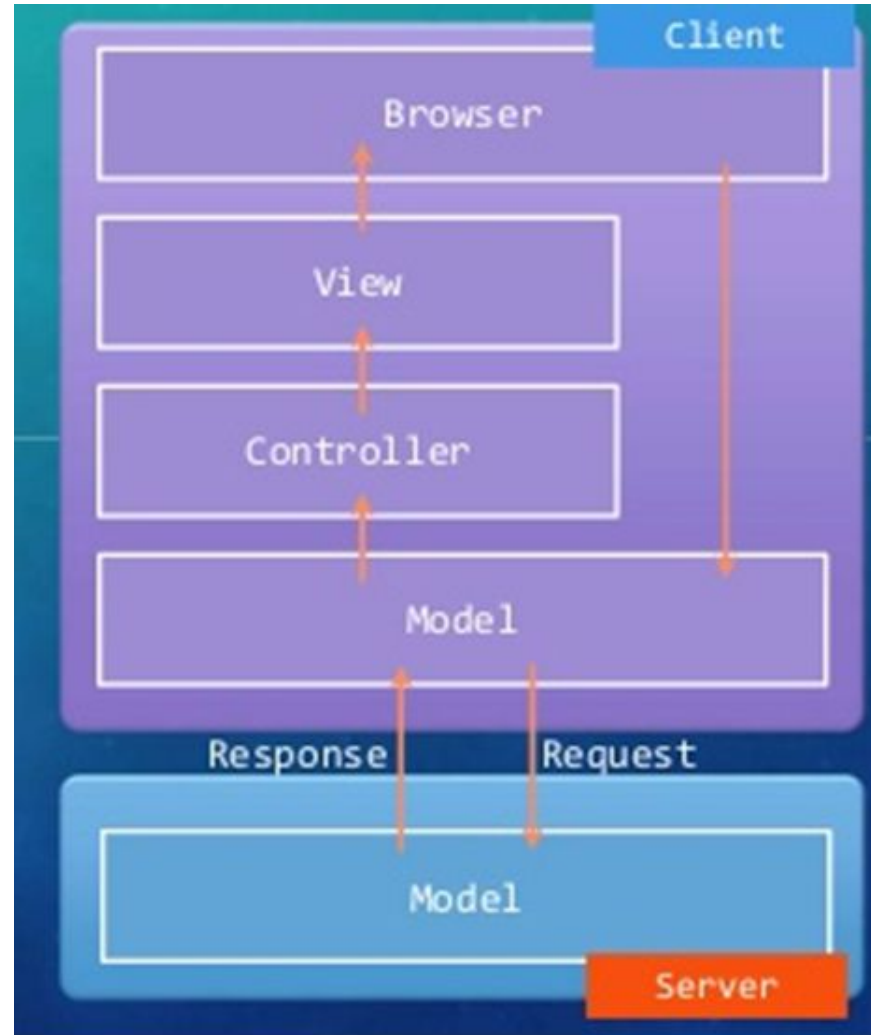
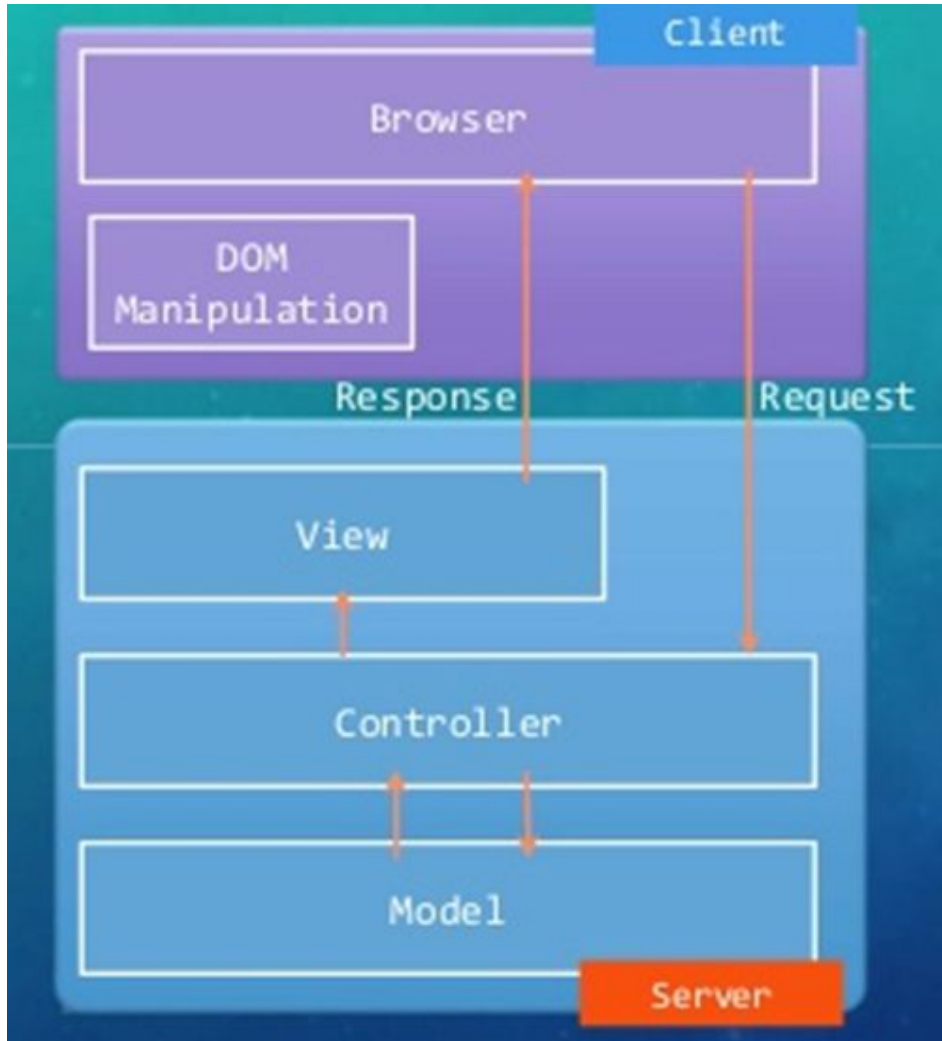
- [Arts and Humanities](#) - [Architecture](#), [Photography](#), [Literature](#)...
- [Business and Economy \[Xtra!\]](#) - [Companies](#), [Investments](#), [Classifieds](#)...
- [Computers and Internet \[Xtra!\]](#) - [Internet](#), [WWW](#), [Software](#), [Multimedia](#)...
- [Education](#) - [Universities](#), [K-12](#), [College Entrance](#)...
- [Entertainment \[Xtra!\]](#) - [Cool Links](#), [Movies](#), [Music](#), [Humor](#)...
- [Government](#) - [96 Elections](#), [Politics \[Xtra!\]](#), [Agencies](#), [Law](#), [Military](#)...
- [Health \[Xtra!\]](#) - [Medicine](#), [Drugs](#), [Diseases](#), [Fitness](#)...
- [News and Media \[Xtra!\]](#) - [Current Events](#), [Magazines](#), [TV](#), [Newspapers](#)...
- [Recreation and Sports \[Xtra!\]](#) - [Sports](#), [Games](#), [Travel](#), [Autos](#), [Outdoors](#)...
- [Reference](#) - [Libraries](#), [Dictionaries](#), [Phone Numbers](#)...



MVC

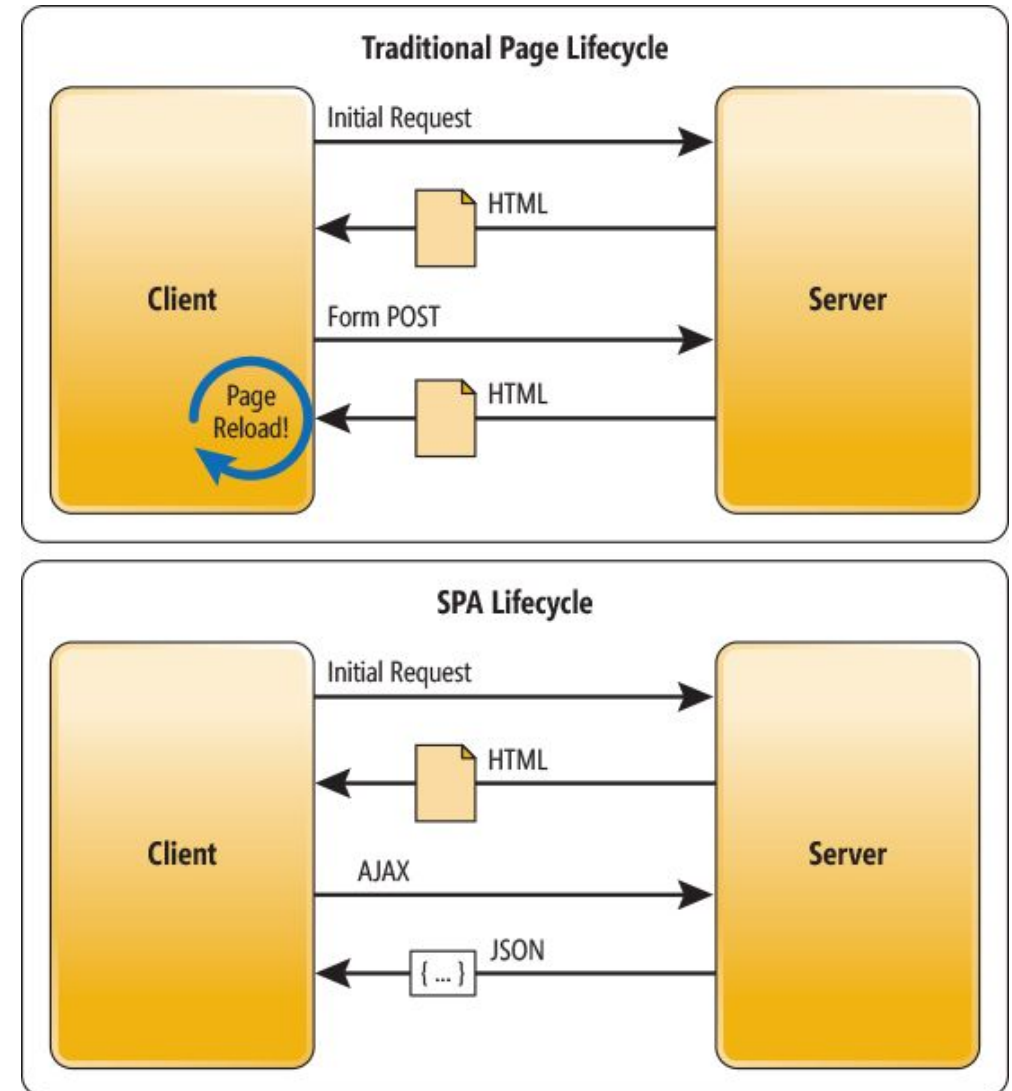


Server side MVC and Client side MVC



Single Page Application (SPA) vs MPA

- SPA interacts with the user by dynamically rewriting the current page rather than loading entire new pages from a server
- appropriate resources are dynamically loaded and added to the page as necessary, usually in response to user actions
- SPA needs a Client-side routing that allows you to navigate around a web page
- SPA offers native application like experience



Single Page Application

- Advantage

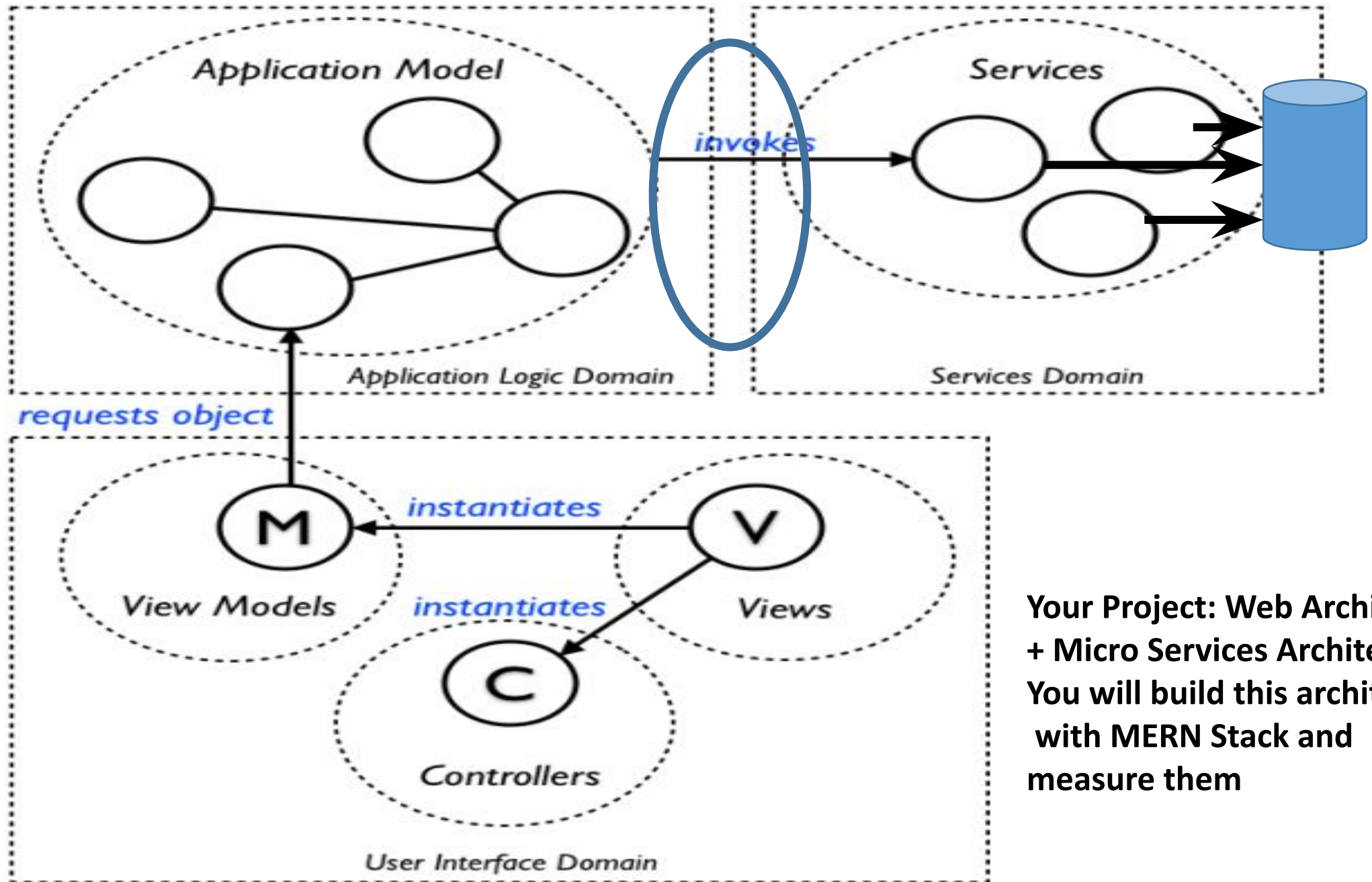
- Single Page Apps are smooth and fast.
- They are easier to develop, deploy and debug.
- Can be transited to mobile apps by reusing the same backend code.

- Disadvantages

- SPAs perform poor on the search engine. But now, with isomorphic rendering/server-side rendering even SPAs can be optimized for search engine too.
- They are less secure compared to traditional multi-page apps because of its cross-site scripting.

SPA and Client side framework

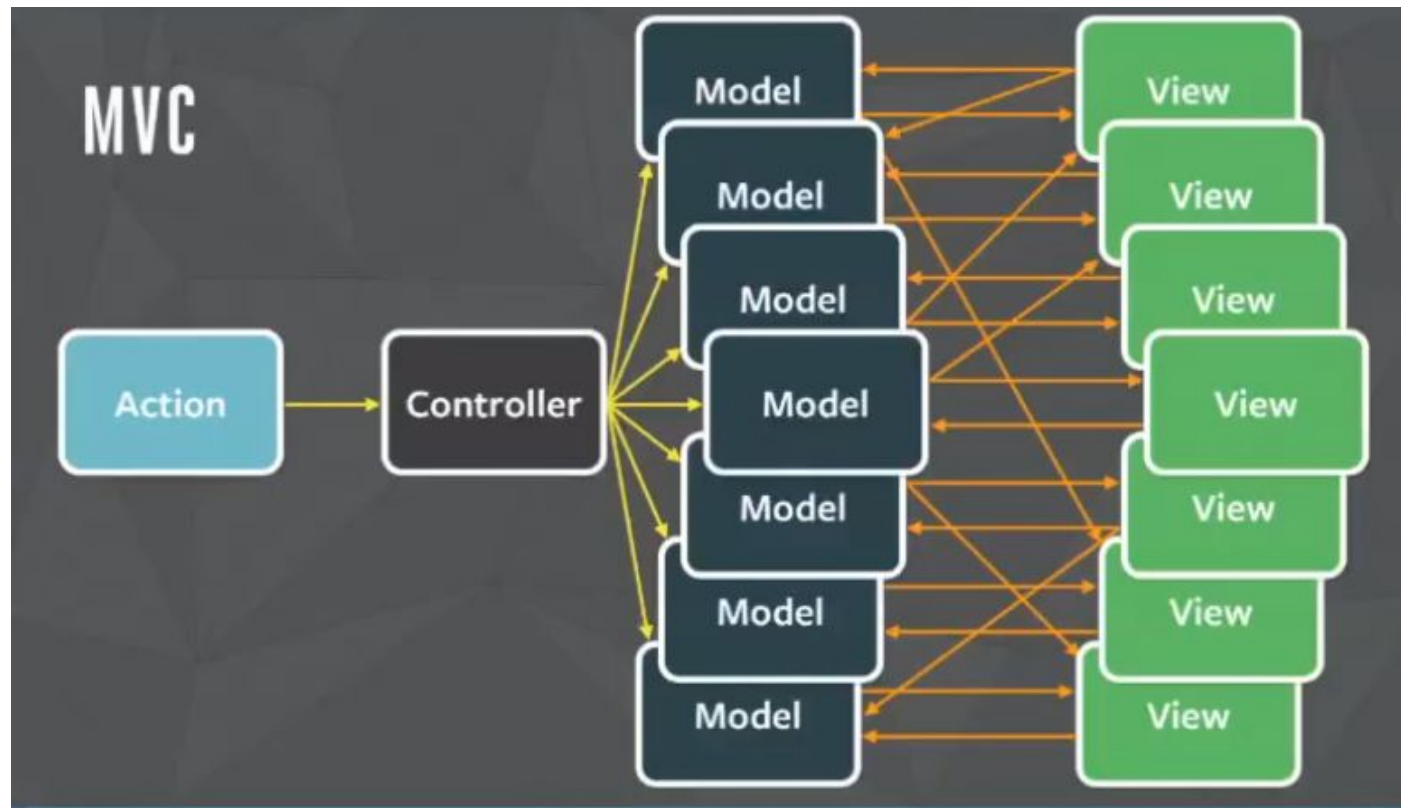
- Framework can manage large, complex application
- Client-side frameworks
- React, AngularJS, Backbone, Ember, ExtJS, Knockout, MetroJS, Vue.JS, etc.
- React
 - UI building library developed by Facebook
 - Using Virtual Dom for Dom manipulation
 - Create the new concepts such as Flux
- AngularJS
 - All-in-one web application framework developed by Google
 - Data binding, Directive, Routing, Security etc



**Your Project: Web Architecture
+ Micro Services Architecture**
You will build this architecture
with MERN Stack and
measure them

Problems with old technology

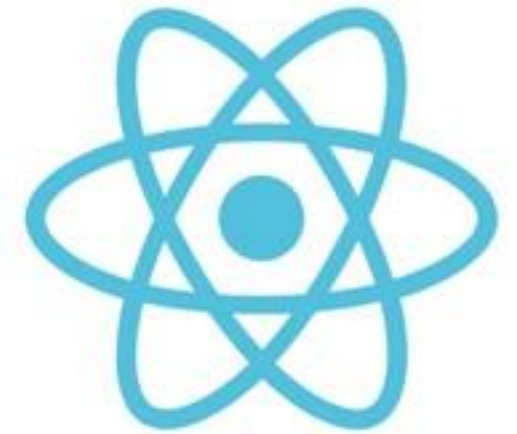
- MVC does not scale
- Typical MVC architecture



Problems with old technology (Contd)

- Complexity of two-way data binding
- Bad UX from using "cascading updates" of DOM tree
- A lot of data on a page changing over time
- Complexity of Facebook's UI architecture

Client-side frameworks



- ReactJS <https://facebook.github.io/react/>
- <https://reactjs.org/>
- **Javascript library for building user interfaces**
- **Components** (collection of HTML, CSS, JS) **are the building blocks of React**
- React is a JavaScript framework that focuses on declarative syntax and virtualization of DOM.
- It is an open-source Javascript library Developed by Meta in 2013.
- It provides a declarative and efficient way to create interactive UI components.
- It allows building more reusable and maintainable UI components with ease.

.

Key concepts

- **JSX:** A syntax extension that allows writing HTML-like code within JavaScript. It makes UI development more readable and easier to manage in React.
- **Components:** The building blocks of a React application. Components are reusable, independent, and can be either functional or class-based.
- **Props:** Read-only data passed from parent components to child components, allowing for dynamic and customizable content
- **State:** An internal data store within a component. When the state changes, the component re-renders to reflect the updates.
- **Unidirectional data flow:** Data in React flows in a single direction, from parent to child components, ensuring predictable and maintainable state management.
- **Virtual DOM:** A lightweight copy of the actual DOM that React uses to efficiently update the UI. It minimizes direct manipulations of the real DOM, improving performance.
- **Hooks:** Built-in functions that allow functional components to use state and other React features without needing class components. Examples include `useState` and `useEffect`.
- **React Router:** A library that enables navigation and routing in React applications, allowing users to move between different views or pages without full-page reloads.

React's popularity stems from its efficiency, reusability, and the large ecosystem of tools and libraries built around it. It is widely used in web development for creating interactive and dynamic user experiences.

Most Forked Repos (Click to View Repo Link on GitHub)

2015

tensorflow/tensorflow	Open source software library for numerical computation using data flow graphs.	4,355	1
facebook/react-native	A framework for building native apps with React.	4,198	2
NARKOZ/hacker-scripts	Based on a true story	3,553	3
apple/swift	The Swift Programming Language	3,068	4

Who uses React?

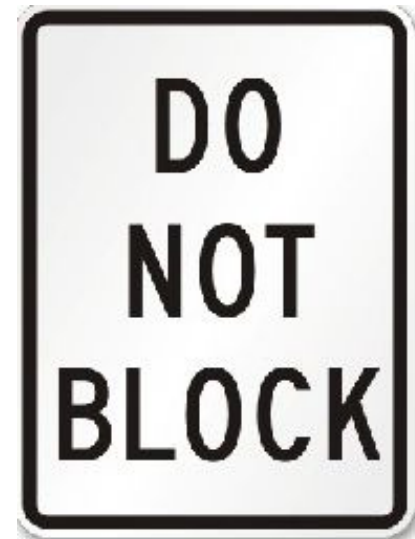
<https://jaydevs.com/top-companies-using-react-js/>

There are too many companies using React JS to list them all here, so instead we're going to cover the top 30 companies that use React JS. We'll then detail what React components they use and the benefit it has for their web and mobile application development.



Why use React?

- Easy to read and understand views
- Concept of components is the *future* of web development
- If your page uses a lot of **fast-updating data** or **real-time data**, React is the way to go



Data Mutation is problem

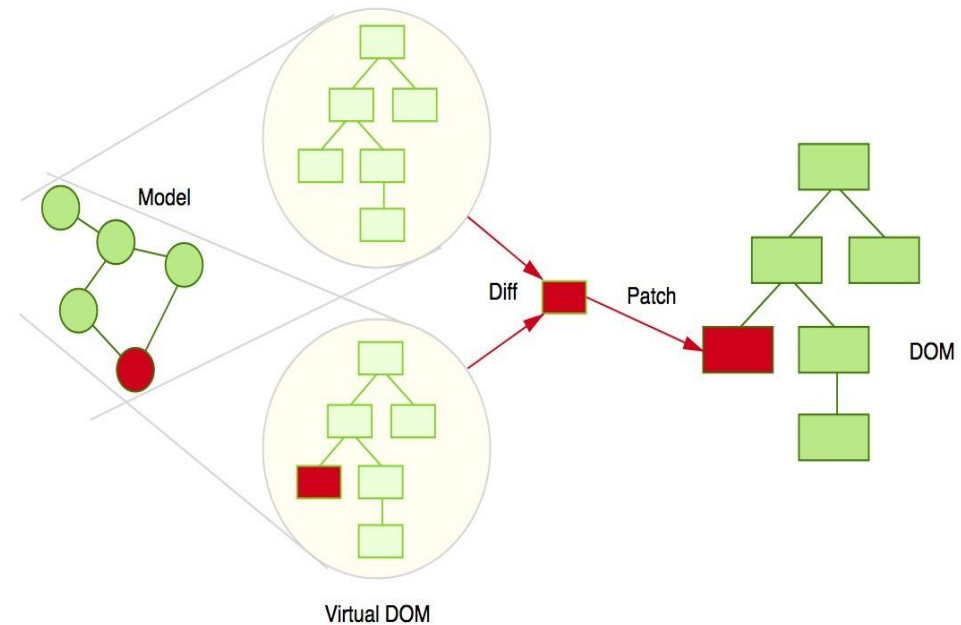
- When data changes, refresh
- When data changes, React re-renders the component
- But Stateful Browser DOM
- Reacts builds new virtual DOM subtrees
- Computes the minimal set of DOM mutations and puts them in a queue and Batch executes all updates



DATA CHANGING OVER TIME IS
THE ROOT OF ALL EVIL

ReactJS

- Virtual DOM. Keeping state of DOM is hard
- Efficient diff algorithm
- Batched update operations
- Efficient update of sub tree only
- Uses observable instead of dirty checking to detect change
- AngularJS uses dirty checking that runs in a cycle after a specified time, Checking the whole model reduces the performance and thus makes the application slow.



React: 50% better performance

MOBILE SITE SEARCH

React rewrite

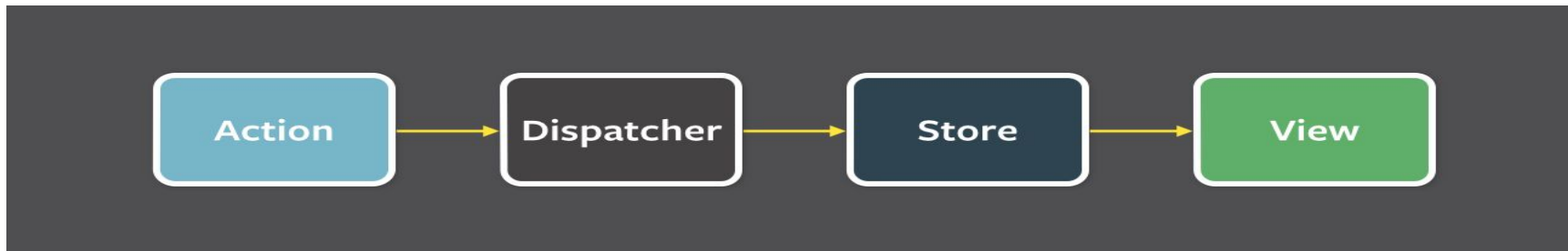
- More features, same amount of time
- With 50% better performance



Source: Facebook

Two-way vs. Unidirectional data flow

- In two-way data binding
 - The view is updated when the state changes, and vice versa.
 - For example, when you change a model in AngularJS, the view automatically reflects the changes.
 - It can lead to cascading updates and changing one model may trigger more updates.
 - View ↔ State
- Unidirectional data flow
 - Mutation of data is done via actions. So, new data always enters into the store through actions.
 - View components subscribe to the stores and automatically re-render themselves using the new data. So, the data flow looks like this :

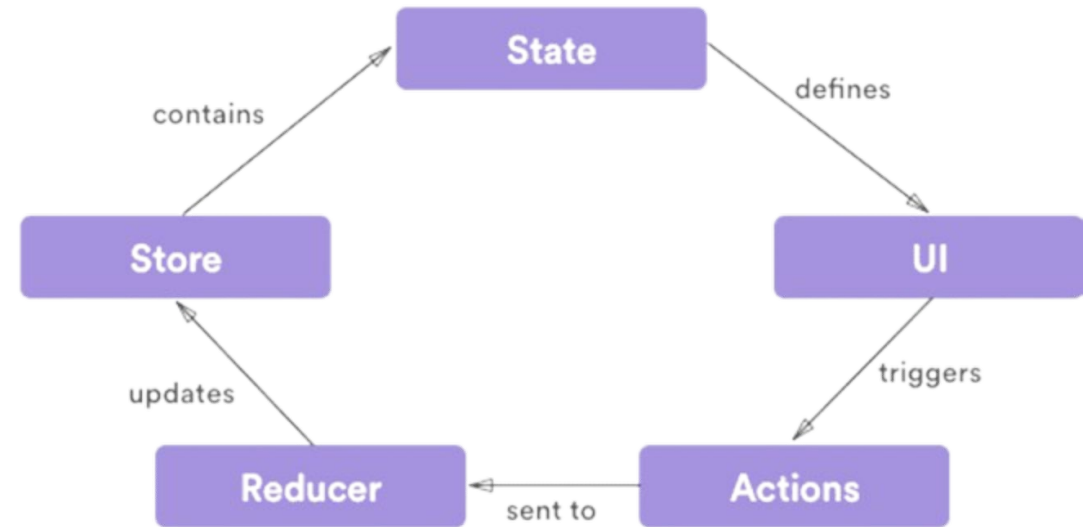


Source: Facebook

React

- **State Management**

- State management is a critical aspect of building complex web applications.
- ReactJS provides several options for managing state Efficiently within its ecosystem.
- Various ways of State Management:
 - Local Component State
 - `useState`
 - Context API
 - Redux



What is a React component?

- ReactJS follows a component-based architecture, where UI are composed of reusable building blocks called components.
- A React component is a JavaScript function or class that returns a JSX (JavaScript XML) representation of the UI.
- It encapsulates the UI logic and state, making it reusable and modular.
- React supports two ways of declaring a component-
- Class Components (old)
- Functional Components (new)

Functional Components (Recommended)

- A simple JavaScript function that returns JSX.
- Can use **Hooks** (`useState`, `useEffect`, etc.) to manage state and lifecycle.
- More concise and easier to understand.

♦ Example: Functional Component with Props & State

```
import { useState } from "react";

function Greeting({ name }) {
  const [message, setMessage] =
    useState("Welcome!");

  return (
    <div>
      <h1>Hello, {name}!</h1>
      <p>{message}</p>
      <button onClick={() => setMessage("Have a great
day!")}>Click Me</button>
    </div>
  );
}

export default Greeting;
```

Why Functional Components are Preferred Over Class Components in React?

1. Simplicity & Readability

- Cleaner, easier to read & write
- No need for `this` or complex class-based syntax

2. Reusability & Modularity

- Small, composable functions improve code maintainability
- Encourages better separation of concerns

3. Improved Testability

- Pure functions ensure predictable output
- Easier unit testing with consistent results

4. Better Performance

- No class overhead, reducing memory usage
- Optimized by React's rendering process

5. Hooks

- `useState`, `useEffect`, and custom hooks bring powerful features
- Enables state management in functional components

6. Conciseness & Maintainability

- Less code, fewer bugs
- Simplifies state & lifecycle management

Class Components Still Exist: Useful in legacy codebases, but modern React favors functional components with hooks for cleaner, more efficient development.

Structure of React Component

- State
 - State represents mutable data that belongs to a component. Unlike props, state can change over time, triggering a re-render of the component.
 - Functional components can also have state using React hooks like `useState`.

State in Functional Components (Modern Approach using Hooks)

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </div>
  );
}

export default Counter;
```

Structure of React Component

- Styling
 - Styling can be applied to React components using CSS classes, inline styles, or CSS-in-JS libraries.
 - CSS classes can be added using the className attribute in JSX.
 - Inline styles can be applied using the style attribute in JSX.

Using CSS Classes (Recommended for maintainability)

```
import './styles.css';

function StyledComponent() {
  return <h1 className="heading">Hello, World!</h1>;
}

export default StyledComponent;
```

CSS-in-JS (Styled Components) allow writing CSS directly inside JavaScript files

```
import styled from "styled-components";

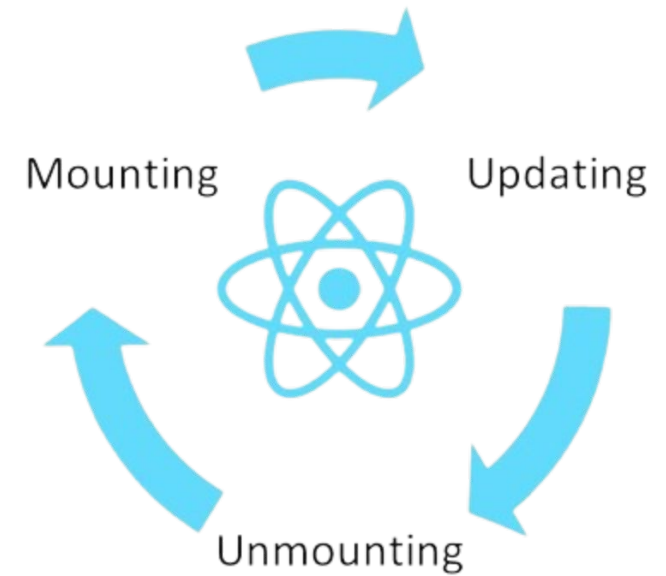
const StyledButton = styled.button`
  background-color: blue;
  color: white;
  padding: 10px;
  border-radius: 5px;
`;

function App() {
  return <StyledButton>Click Me</StyledButton>;
}

export default App;
```

Component Life Cycle

- React components have a life cycle consisting of different phases and methods that are executed at s
 - The mounting phase is when a new component is created and inserted into the DOM.
 - The updating phase is when the component updates or re-renders. This reaction is triggered when the props are updated or when the state is updated.
 - The last phase within a component's lifecycle is the unmounting phase, when the component is removed from the DOM.



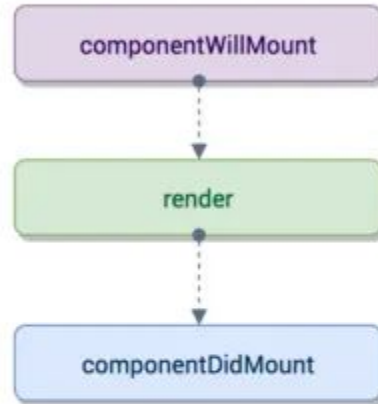
Life Cycle Hooks

- `componentDidMount()`
 - This hook is called after a component is mounted (rendered for the first time) in the DOM.
- `componentDidUpdate(prevProps, prevState)`
 - This hook is called after a component is updated and re-rendered in response to changes in props or state.
- `componentWillUnmount()`
 - This hook is called just before a component is unmounted and removed from the DOM.
- `shouldComponentUpdate(nextProps, nextState)`
 - This hook is called before a component is re-rendered and allows you to control if the re-rendering should occur.

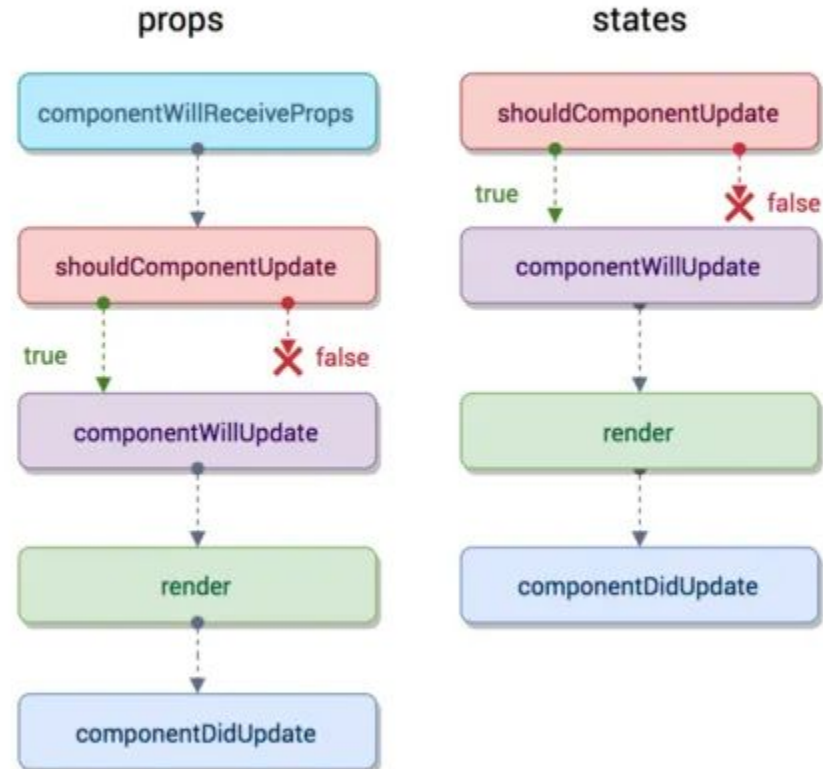
Initialization

setup props and state

Mounting



Updation

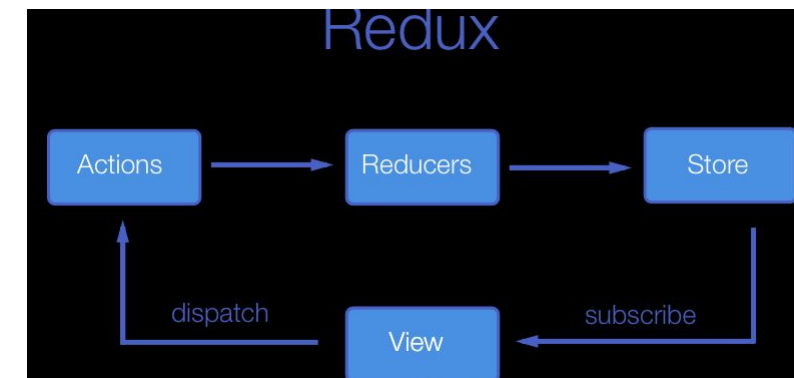
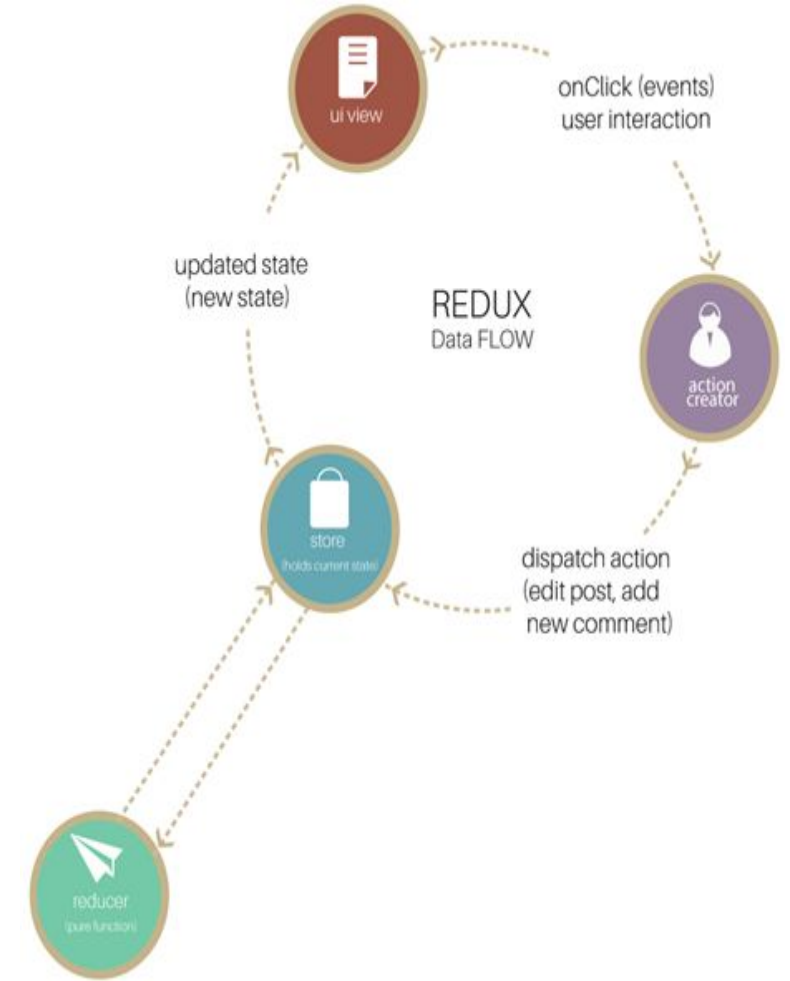


Unmounting



Redux

- Redux is a state container for JavaScript apps, often called a Redux *store*. It stores the whole state of the app in an immutable object tree.
- Actions: Actions are how the application interacts with the store. The application sends an action with some attached data. These actions are then handled by a so-called **reducer**.
- Store: This structure stores the complete state of our application. Given only the content of the store, the application should be able to save and recover the current state of the application
- Reducer: The Reducer is a function that takes the current state and an action and transforms it to a new state.



Redux

- The state is read-only: The state of the application can only be changed via actions. This is very useful for debugging since the changing state flow is much more clear.
- The Redux store API is tiny and has only four methods:
 - `store.getState()`—Returns the current state object tree.
 - `store.dispatch(action)` - Dispatch an action to change the state.
 - `store.subscribe(listener)` - Listen to changes in the state tree.
 - `store.replaceReducer(nextReducer)` - Replaces the current reducer with another. This method is used in advanced use cases such as code splitting

Simple Redux Tutorial: <https://appdividend.com/2017/08/23/redux-tutorial-example-scratch/>

JSX (JavaScript XML)

- JSX (JavaScript XML) is a syntax extension for
- JavaScript is used in React.
- JSX allows you to write HTML-like code within JavaScript, making it easier to create and manipulate the UI.
- JSX allows embedding JavaScript expressions within curly braces {}.
- JSX needs to be compiled into plain JavaScript to be understood by the browser.

```
class HelloWorld extends React.Component {  
  render() {  
    return (  
      React.createElement(  
        'h1',  
        {className: 'large'},  
        'Hello World'  
      )  
    );  
  }  
}
```

<https://reactjs.org/tutorial/tutorial.html>

<https://www.w3schools.com/REACT/DEFAULT.ASP>

 **React**

Docs

Tutorial

Community

Blog

 Search docs...

GitHub

v15.6.1

TUTORIAL

[Before We Start](#)

[What We're Building](#)

[Prerequisites](#)

[How to Follow Along](#)

[Help, I'm Stuck!](#)

[Overview](#)

[What is React?](#)

[Getting Started](#)

[Passing Data Through Props](#)

[An Interactive Component](#)

[Developer Tools](#)

[Lifting State Up](#)

[Why Immutability Is Important](#)

[Functional Components](#)

[Taking Turns](#)

[Declaring a Winner](#)

[Storing A History](#)

[Conclusion](#)

Tutorial: Intro To React

[Edit on GitHub](#)

Before We Start

What We're Building

Today, we're going to build an interactive tic-tac-toe game.

If you like, you can check out the final result here: [Final Result](#). Don't worry if the code doesn't make sense to you yet, or if it uses an unfamiliar syntax. We will be learning how to build this game step by step throughout this tutorial.

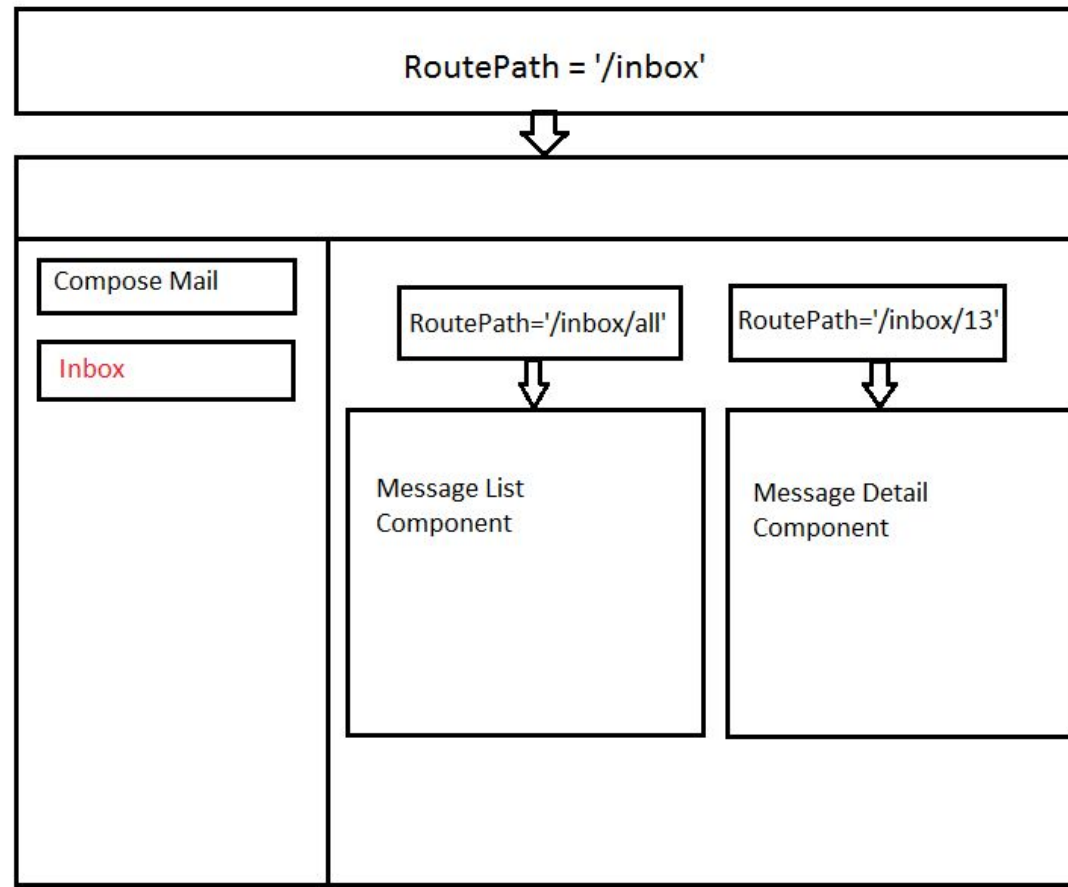
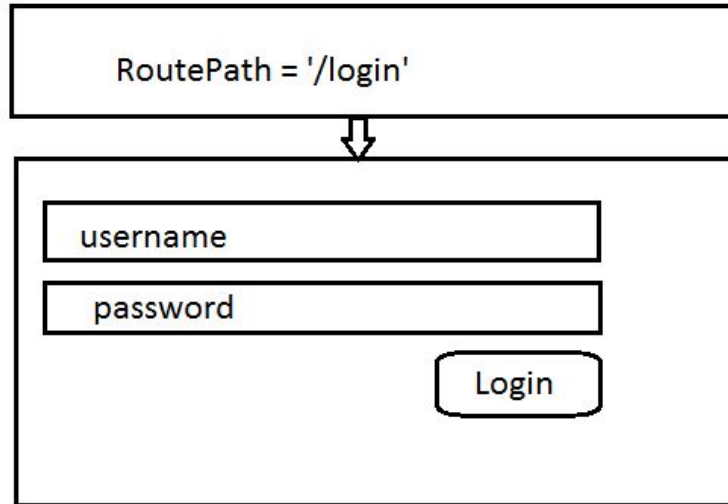
Try playing the game. You can also click on a link in the move list to go "back in time" and see what the board looked like just after that move was made.

Once you get a little familiar with the game, feel free to close that tab, as we'll start from a simpler template in the next sections.

Prerequisites

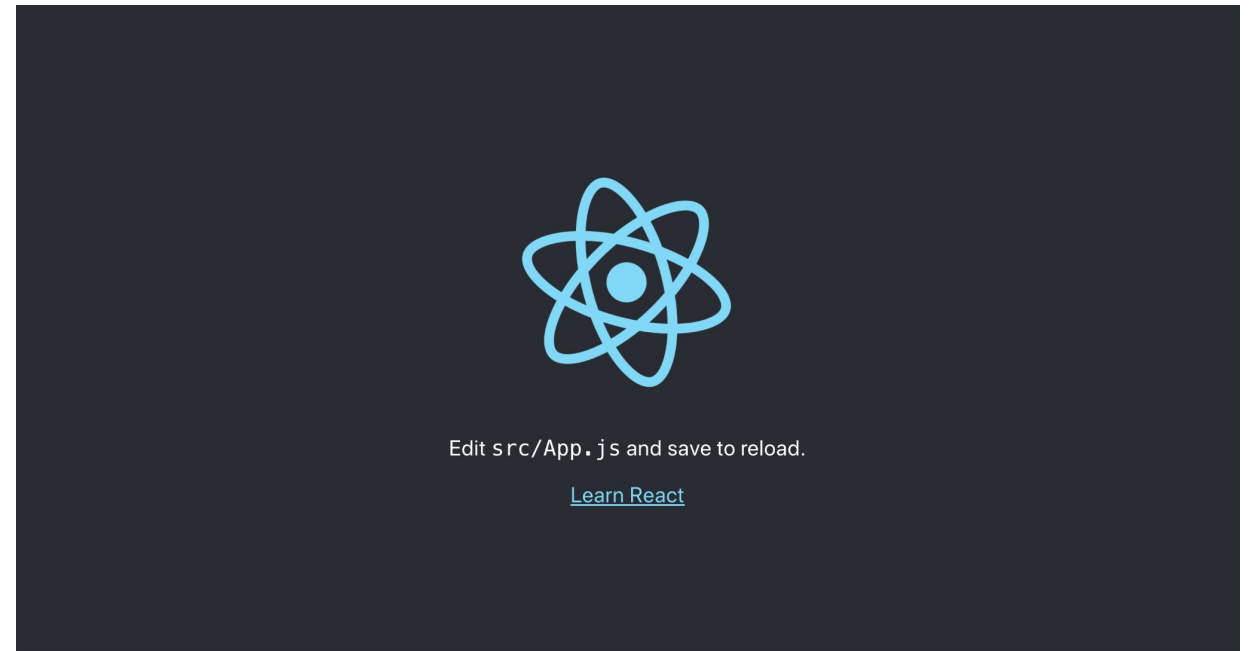
We'll assume some familiarity with HTML and JavaScript but you should be able to follow along even if you haven't used them before.

Composition of Components



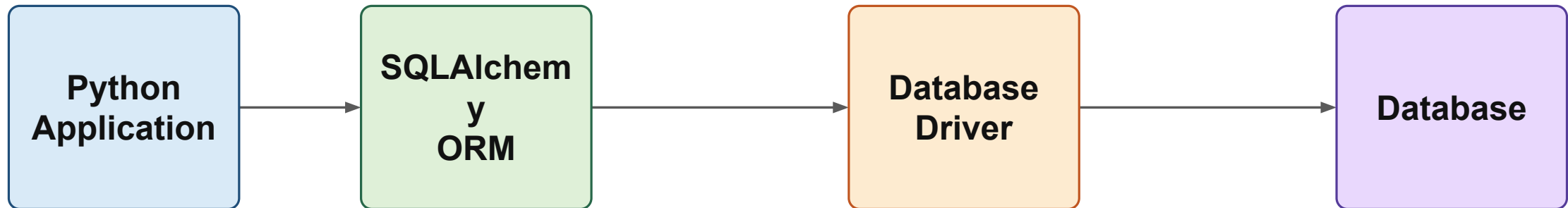
Running a React Application

- To run a React Application, we need to install
 - NodeJS
 - Node Package Manager (NPM)
- Create React Application
 - `npx create-react-app my-app`
- Run development server
 - `cd my-app`
 - `npm start`
- Build Project
 - `npm run build`



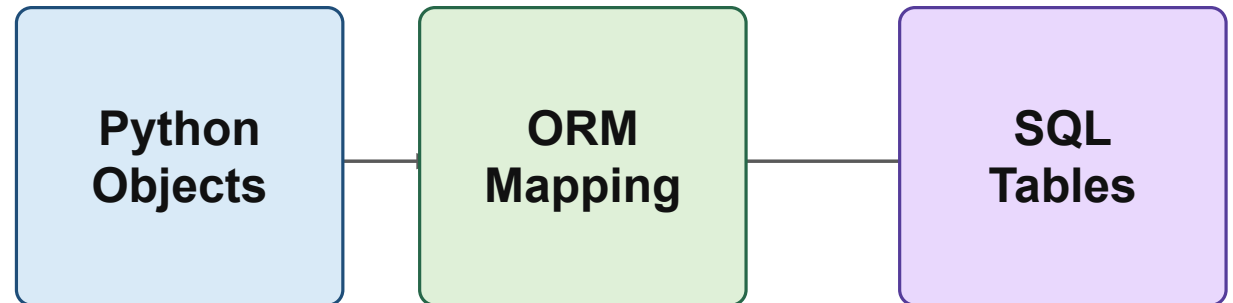
SQLAlchemy Introduction

Engine • Session • Model • ORM • CRUD



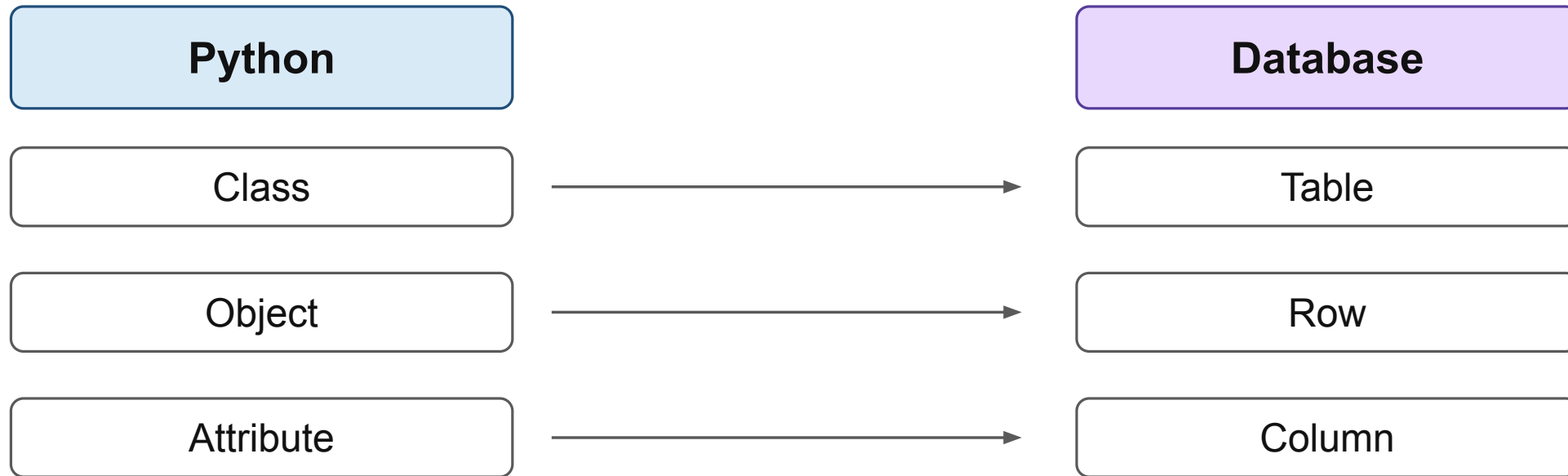
The problem SQLAlchemy solves

- Python programs work with objects
- Databases store rows in tables
- Raw SQL is powerful, but code can become repetitive
- SQLAlchemy creates a bridge between Python and SQL



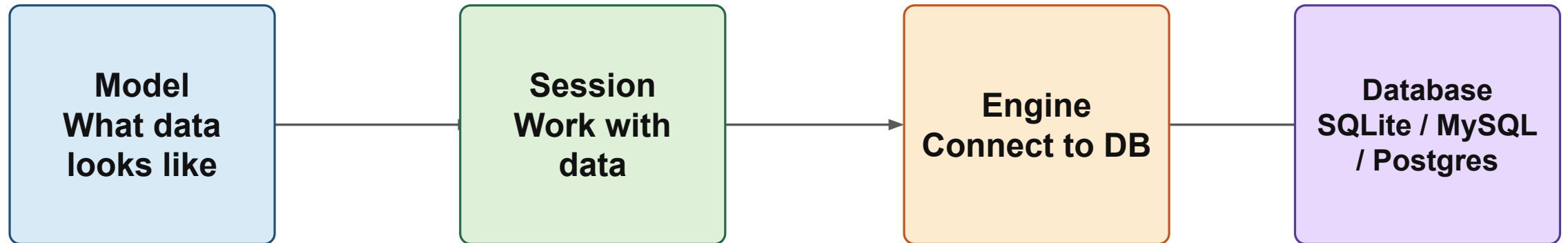
What is ORM?

ORM = Object Relational Mapping



Example: Student class ↔ students table

The three core pieces

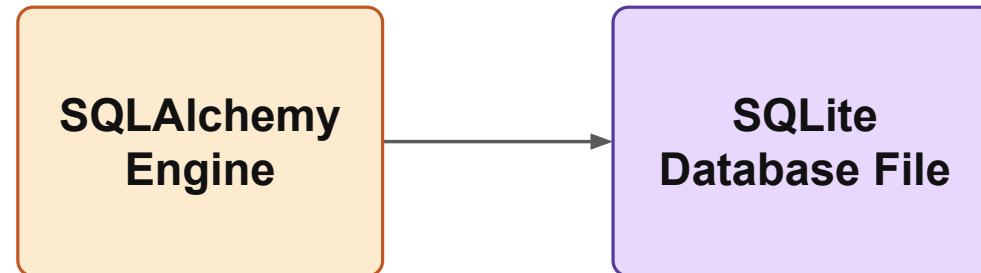


Beginner shortcut: Model = table blueprint, Session = work area, Engine = database connection

Engine: connection to the database

- Created with `create_engine()`
- Knows the database URL
- Manages low-level database connections
- Usually created once when the app starts

```
engine = create_engine(  
    "sqlite:///school.db"  
)
```



Model: Python class for a database table

```
class Student(Base):  
    __tablename__ = "students"  
  
    id = mapped_column(primary_key=True)  
    name = mapped_column(String(100))  
    major = mapped_column(String(100))
```



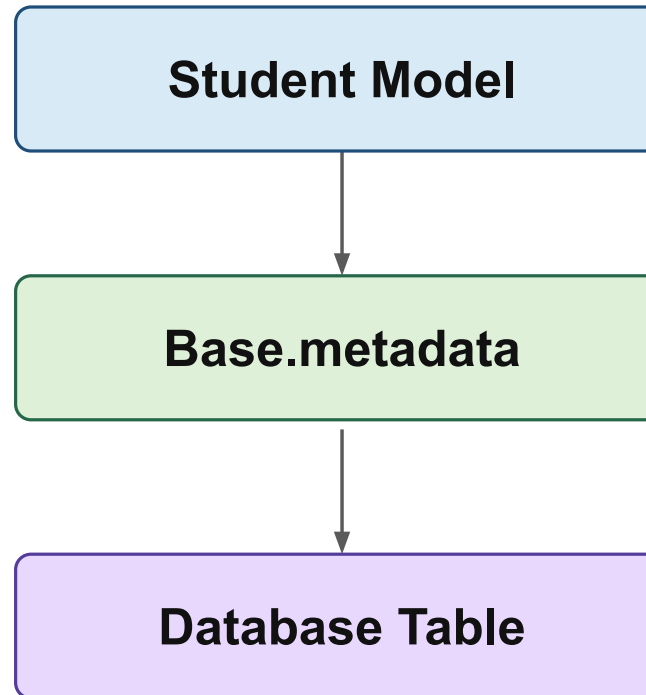
students table

id
name
major

Model definitions are the “map” between Python and table structure.

Base and metadata: create tables

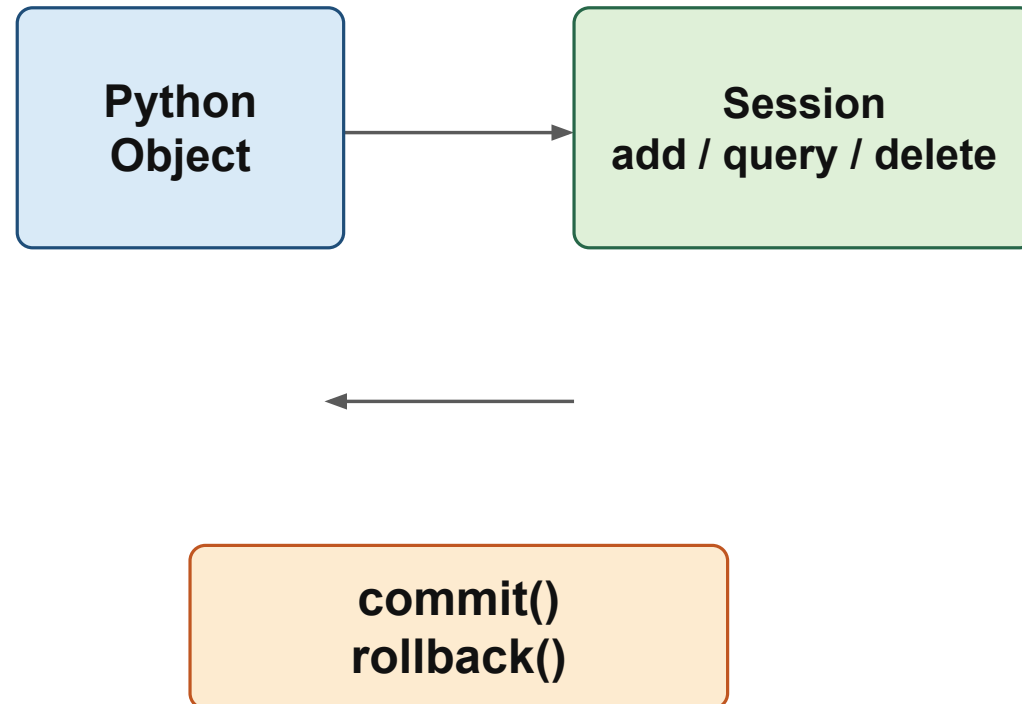
- Base is the parent class for ORM models
- Models add table information to Base.metadata
- `create_all(engine)` creates missing tables



```
Base.metadata.create_all(engine)
```

Session: working conversation with the database

- The Session is where ORM work happens
- It tracks new, changed, and deleted objects
- `commit()` saves the transaction
- `rollback()` cancels the transaction



Create: insert one row

```
with Session(engine) as session:
    student = Student(
        name="Alice",
        major="Computer Science"
    )

    session.add(student)
    session.commit()
```

Student(...)
Python object

session.add()

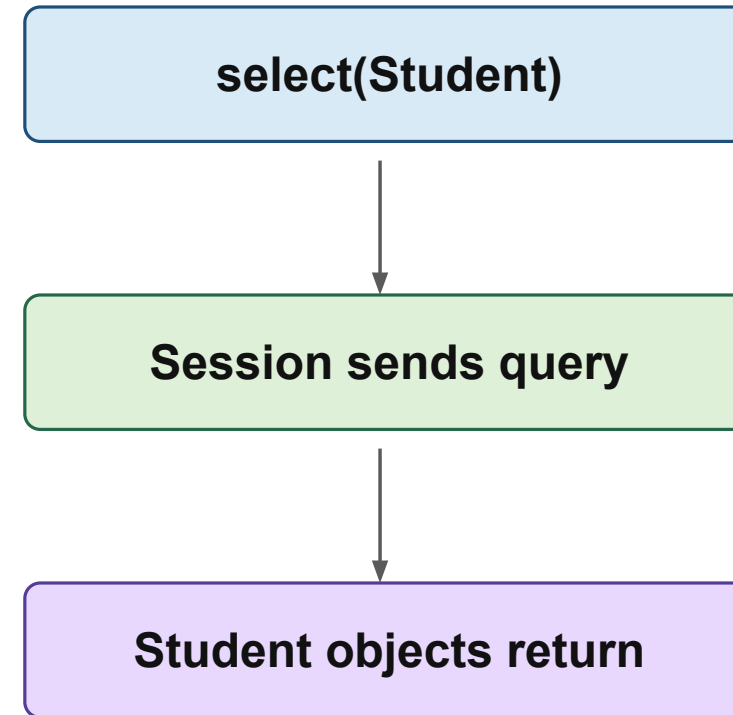
session.commit()
row saved

Read: select rows

```
stmt = select(Student)

with Session(engine) as session:
    students = session.scalars(stmt).all()

for student in students:
    print(student.name)
```



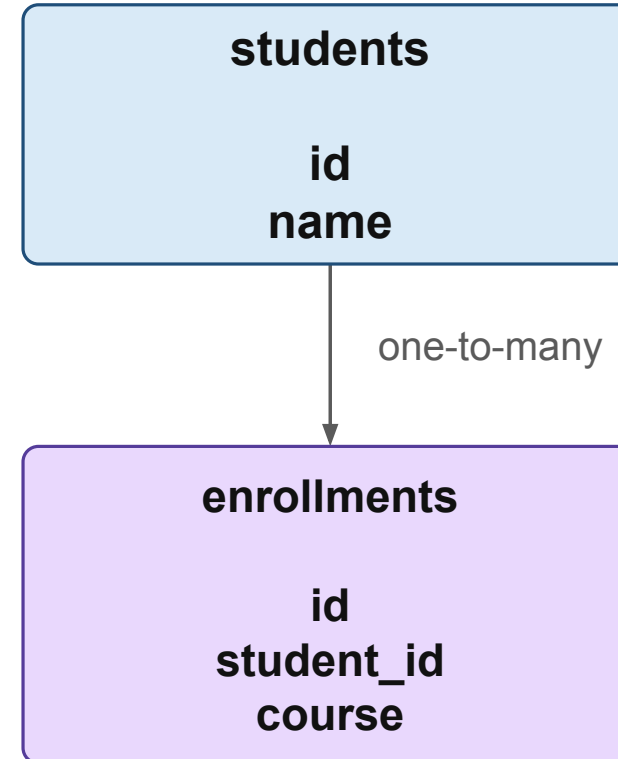
CRUD overview

Action	SQLAlchemy	Database meaning
Create	<code>session.add(obj)</code>	INSERT row
Read	<code>select(Model)</code>	SELECT rows
Update	<code>obj.field = value</code>	UPDATE row
Delete	<code>session.delete(obj)</code>	DELETE row
Save	<code>session.commit()</code>	COMMIT transaction

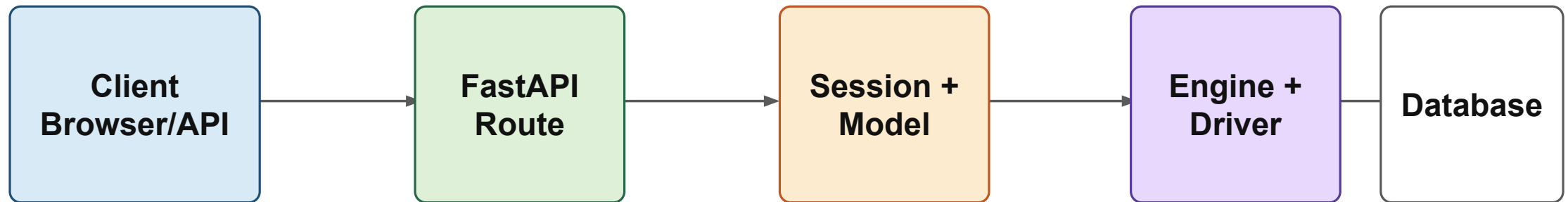
Commit is the point where changes become permanent.

Relationships: tables connected to tables

- Real databases usually have multiple related tables
- Example: one student can have many enrollments
- SQLAlchemy models can describe those relationships
- This lets Python navigate related data more naturally

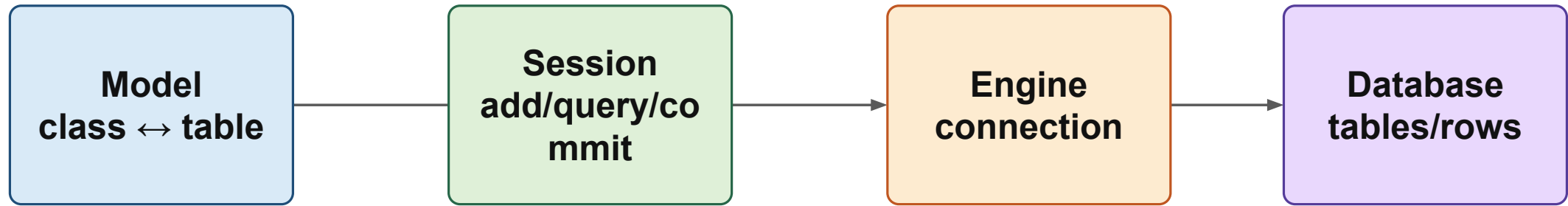


Where SQLAlchemy fits in a web app



Most web apps create a short-lived session for each request.

Summary



The key idea: SQLAlchemy lets Python code work with database data as objects.

Next topic: relationships, joins, migrations with Alembic, and FastAPI dependency injection.

Prompt Engineering

Prompt Engineering— Definition

Prompt engineering is the practice of designing the input text (and optional parameters) that steer an LLM's behavior toward a specific goal. It focuses on *what you provide the model*: clear task framing, constraints, desired style, output format, and any evaluation cues.

Prompt formula:

Task + Constraints + Output Format (+ *Hints / Examples*)

Why it matters:

- Improves output quality and reliability.
- Supports evaluation and safety (e.g., grounding, refusals, format control).
- Is inherently iterative—effective prompts usually require testing and refinement.

The Prompt (single instruction)

A **prompt** contains just enough to perform one task.

Summarize this article in 3 bullet points, focusing on the causes.

Strengths: fast, minimal tokens.

Limits: ambiguity, higher variance, no examples, no memory.

Elements of a Prompt

A prompt = Instructions + (Context) + Input data + Output indicator.

- **Instructions**
what to do
- **Context**
optional background
- **Input data**
the item to act on
- **Output indicator**
where/how to answer

Example

Instructions

text into neutral, negative or positive

Input data

Text: I think the food was okay.

Output indicator

Sentiment:

Context is optional here — add it when the model needs background (notes, a document, prior turns).

A practical prompt template

Use this when reliability matters

Role: who the model is in this interaction

Task: what success looks like

Inputs: what data is provided

Constraints: policies, scope, citations

Output format: exact schema / headings

Examples: few-shot for tricky cases

Template

SYSTEM: You are a helpful TA for an LLM course.

TASK: Answer the question using ONLY the provided notes.

CONSTRAINTS:

- If the notes don't contain the answer, say: "Not in notes."
- Keep the response ≤ 6 bullets.

FORMAT: bullets

NOTES:

{context}

QUESTION:

{question}

Prompt anatomy: seven parts

A checklist for any serious prompt — each part answers one question.

ELEMENT	QUESTION IT ANSWERS	EXAMPLE
Role	What perspective?	“You are a support drafting assistant.”
Task	What must it produce?	“Draft a reply and an escalation note.”
Context	What facts/policies apply?	“Use only the ticket and approved help center.”
Constraints	What must it avoid?	“Don’t promise refunds or invent behavior.”
Process	How to reason?	“Extract facts, check gaps, draft, self-check.”
Output format	What structure?	“Summary, reply, sources, confidence, escalation.”
Quality checks	How to self-verify?	“Validate against constraints; revise if needed.”

Few-shot prompting (teach the pattern)

Provide 1–3 high-quality examples of the desired behavior.

Examples are most helpful for: edge cases, tone/style, and strict formatting.

Avoid huge examples—use minimal, representative cases.

If you need strict JSON, consider constrained decoding instead.

Example

User: Classify sentiment.

Text: "The update broke my workflow."

Answer (JSON):

```
{"sentiment": "negative", "confidence": 0.9}
```

User: Classify sentiment.

Text: "It works, nothing special."

Answer (JSON):

```
{"sentiment": "neutral", "confidence": 0.6}
```

User: Classify sentiment.

Text: "{text}"

Answer (JSON):

Structured output & constraints

Specify a schema (JSON) or a rigid format (headings).

Prefer machine-checkable outputs for pipelines (agents, RAG).

Combine with: temperature=0–0.2 for stability.

For strict validity:
grammar-constrained sampling /
function calling.

Schema example

Return JSON ONLY:

```
{  
  "answer": string,  
  "citations": [{"source": string,  
    "quote": string}],  
  "uncertainties": [string]  
}
```

LangChain: prompt templates + LangChain Expression Language (LCEL)

- Keep prompts version-controlled (templates, not ad-hoc strings).
- Separate: system instructions vs user content.
- The system message defines the behavior and rules of the AI.
- User message defines the actual task or data supplied by the user
- Use output parsers (JSON) to reduce downstream errors.

Example

```
from langchain_core.prompts import
ChatPromptTemplate
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini",
temperature=0)

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a concise tutor. Output
JSON only."),
    ("user", "Explain {topic} in 5 bullets.
Return JSON with keys: bullets.")
])

chain = prompt | llm
result = chain.invoke({"topic": "KV cache"})
```

The `|` symbol means: Take the output of the component on the left and send it to the component on the right.

Context engineering

What the model sees: retrieval, memory, tools,
and token budgets

Context Engineering — Definition

Context Engineering designs everything the model sees at inference time, not just the prompt: instructions, examples, retrieved documents, tools, memory/state, and control flow.

- Prompt engineering = how you ask
- Context engineering = what you give the model to think with

Prompt vs Context Engineering

Prompt engineering = “How do I ask?”

Instructions & constraints

Examples (few-shot)

Output format

Tone/persona

Decoding params (temp/top_p)

Context engineering = “What do I provide?”

RAG: retrieve the right docs

Chunking + embeddings + ranking

Context

compression/summarization

Memory & conversation state

Tool outputs + schemas

Token budget management

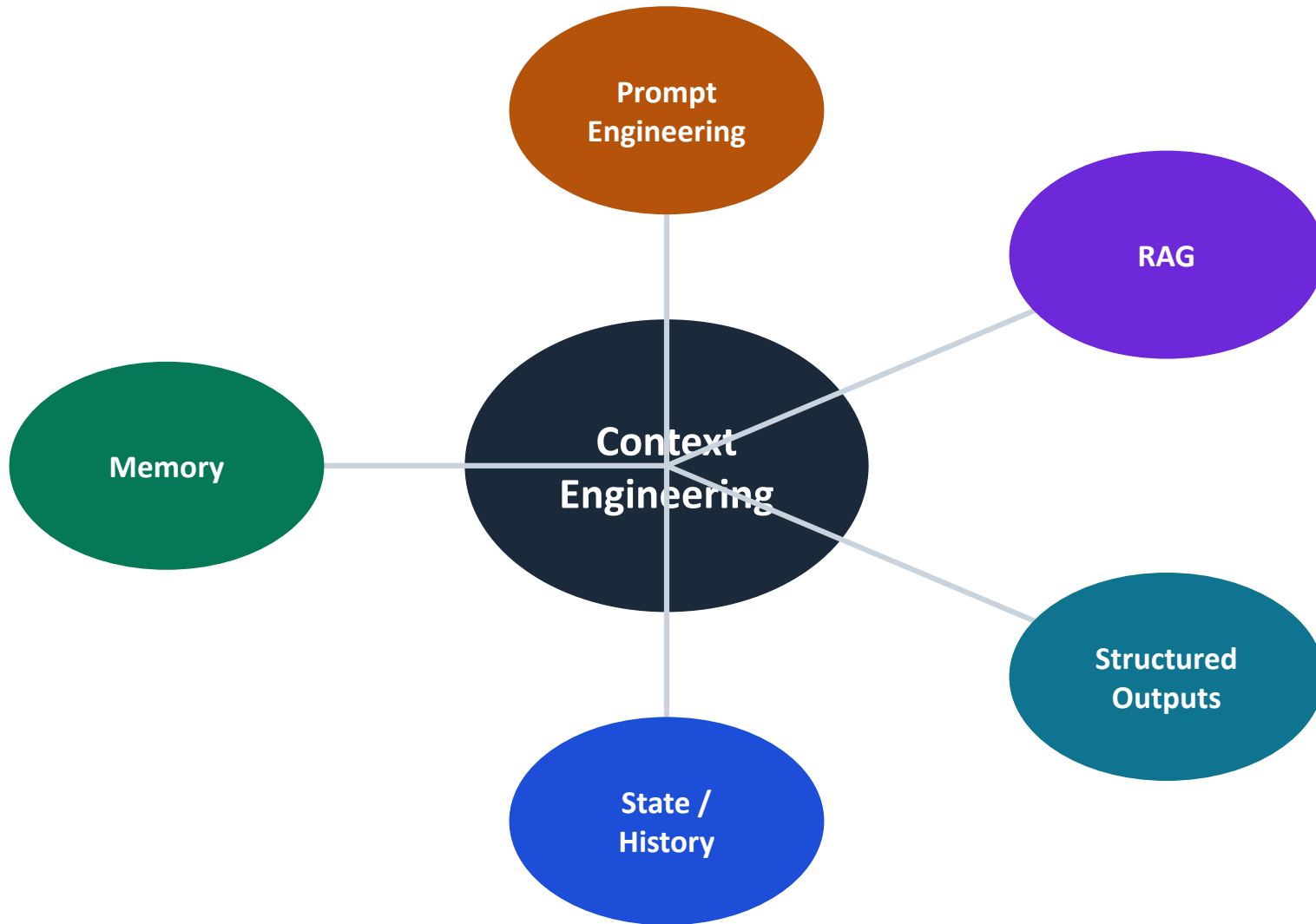
A 5-prompt method to build a grounded kit

Build the system in stages: define the job, ground it, design the process, add controls, package it.

- 1 Frame the task: what the AI is supposed to do**
Purpose, boundaries, required inputs, desired output, and when a human must review.
- 2 Add grounded context: what sources the AI is allowed to use**
Use only approved sources; state freshness & citation; mark missing info; don't invent.
- 3 Design the workflow: turn the task into a process.**
Map intake → approved use; per step: owner, input, action, output, escalation path.
- 4 Define controls & evaluation: How to evaluate the AI is behaving correctly?**
Privacy rules, human-review triggers, a 1–5 scorecard, a release rule, and test cases.
- 5 Assemble the kit: build the actual reusable package**
Production prompt, intake form, output format, checklists, test set — participant-ready.

Everything the model sees is “context engineering”

Prompt engineering is one input; context engineering assembles all of them.



What goes into context:

- Prompt — instructions & format
- RAG — retrieved documents
- Memory — prefs, past turns
- State/History — plan, checklist
- Structured outputs — schema

All of it shares one scarce budget: the context window.

Core Components of Context Engineering

- **Prompts** (user instructions and system directives)
- **Retrieval** (documents from RAG systems)
- **Tool calls** (API responses and external data)
- **Memories** (past conversations and learned preferences)
- **Few-shot examples** (training demonstrations)
- **Structured outputs** (format and schema instructions)

RAG: Overview & Architecture

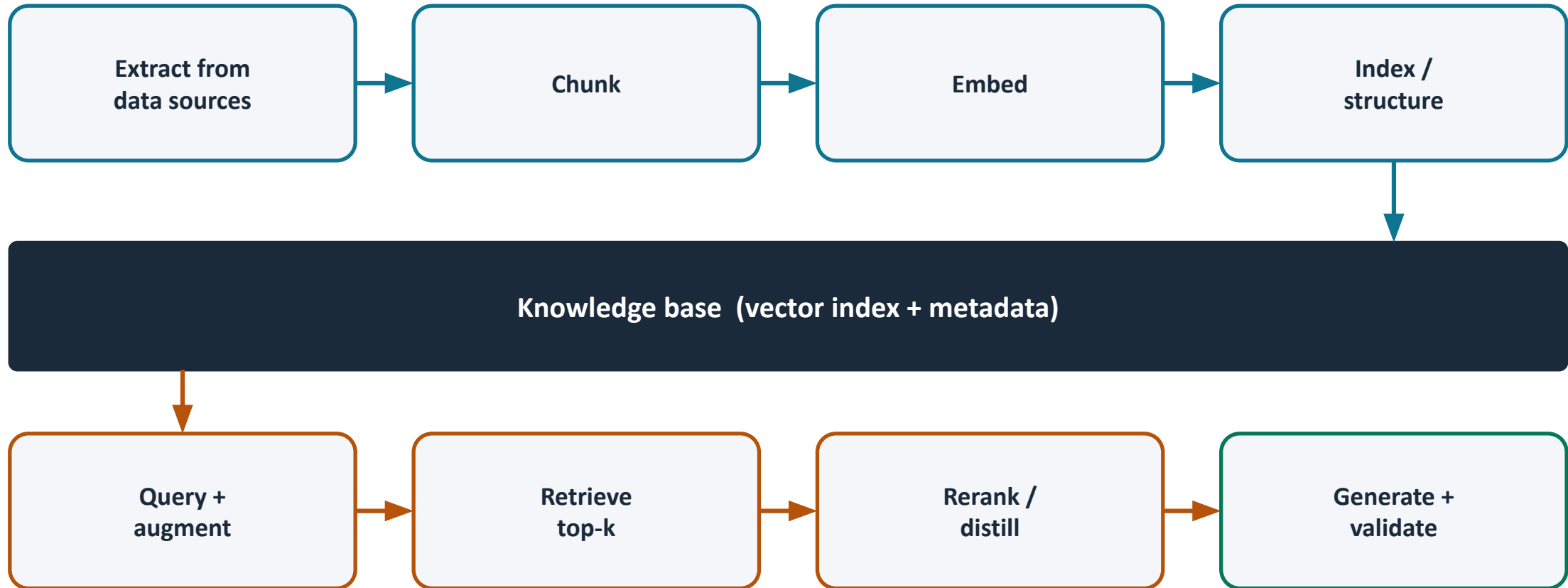
Definition: combine a parametric model with a non-parametric memory (retriever + index) so outputs can be grounded in external, up-to-date knowledge.

Two stages: (1) Indexing (ingest $\rightarrow$ chunk $\rightarrow$ embed $\rightarrow$ store) and (2) Retrieval + generation (fetch top-k, then prompt an LLM).

Why it helps: improves specificity and factuality vs. parametric-only models.

A RAG system: from data to grounded answer

Indexing (top row) is done once; retrieval + generation (bottom row) runs per query.



Quality hinges on retrieval: wrong chunks → hallucinations. Reranking and compression usually help more than a bigger prompt; require citations.

Distillation

Reranking: chooses the best chunks. Which retrieved documents or chunks are most relevant to this question?

Context Distillation: What information inside these selected chunks is actually needed?

Filter Summarize, Deduplicate Extract facts, Compact context documents → compact useful context

Knowledge Distillation

large teacher model → smaller student model

Memory & tool outputs are part of “context”

Memory patterns

Short-term: last N turns

Long-term: user prefs, profile

Summarized conversation state

Task state (plan, checklist)

Tool outputs

Normalize tool results (JSON)

Summarize noisy logs

Store key facts + citations

Don't dump raw pages unless needed

RAG (Retrieval-Augmented Generation) in 3 Steps

1. **Retrieve** top-k passages for the query
2. **Construct** a compact context pack (citations, snippets)
3. **Generate** with grounding, refusing when insufficient evidence

Effect: fewer hallucinations, better factuality; tokens spent on **relevant** info.

Measuring Impact (Rubric)

- **Accuracy** (task-specific)
- **Faithfulness** (to given evidence)
- **Format compliance** (schema, JSON)
- **Efficiency** (tokens, latency)
- **Robustness** (edge cases)
- **Repeatability** (variance across runs)

Track before/after when changing prompts or context kits.

Evaluating prompts & RAG: measures + severity

Track before/after when you change a prompt or the retrieved context.

What to measure

Task success rate — Did the AI complete the entire task correctly? $\text{cases meeting every must-pass} \div \text{total}$

Grounded accuracy — supported by the evidence $\text{supported claims} \div \text{all material claims}$

Escalation recall — AI sends risky cases to a human $\text{high-risk cases escalated} \div \text{all high-risk}$

Reviewer effort — How much human work is still required? $\text{median review} + \text{correction time}$

Cost per approved output — whether the workflow is economically useful, $\text{model} + \text{tool cost} \div \text{approved results}$

Failure severity → action

Critical: privacy breach, unsafe action, fabricated commitment → stop the pilot

High: material wrong claim / bad calc / missed clause → reject & revise

Medium: incomplete coverage or heavy reviewer edits → track & improve

Low: style / formatting / clarity → fix in normal iteration

Release gate: minimum quality condition, ship only above a set score with no Critical/High failures.

- **LangChain** and **LlamaIndex**. Both frameworks are designed to handle document ingestion, splitting, indexing, and chaining together steps for seamless RAG workflows
- LlamaIndex → handle ingestion, indexing, & retrieval (its sweet spot for RAG).
- LangChain → handle orchestration/agents, prompts, and tool use around that retriever.

- **LangChain**

- Specializes in **orchestration and chaining** of LLM calls.
- Provides abstractions for **prompts, tools, agents, memory, and workflows**.
- Lets you combine LLMs with other software systems (search engines, APIs, calculators, etc.) into multi-step reasoning chains.

RAG Quality Levers (retrieval side)

- **Embeddings model:** pick for domain/language; better sentence encoders improve top-k recall and margins (dimensionality $\neq$ quality).
- **Chunking strategy:** sentence/recursive for coherence; token for strict budget;
- **semantic** for content-aware splits; hierarchies for multi-scale retrieval.
- **Retriever settings:** k, metric (cosine/dot/L2), **MMR/diversity**, and optional
- **rerankers** (cross-encoders) for higher precision.

LangChain: simple RAG with LCEL

Minimal pattern

```
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough

# 1) Build/store the index (offline)
db = FAISS.from_texts(docs, embedding=OpenAIEmbeddings())
retriever = db.as_retriever(search_kwargs={"k": 6})

# 2) Compose a RAG chain (online)
prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer using ONLY the context. Cite sources as [#]."),
    ("user", "Context:\n{context}\n\nQuestion: {question}")
])
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

rag = {"context": retriever, "question": RunnablePassthrough()} | prompt | llm
rag.invoke("What is top_p?")
```

LlamaIndex: query engine with citations

Typical pattern

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
from llama_index.core.node_parser import SentenceSplitter

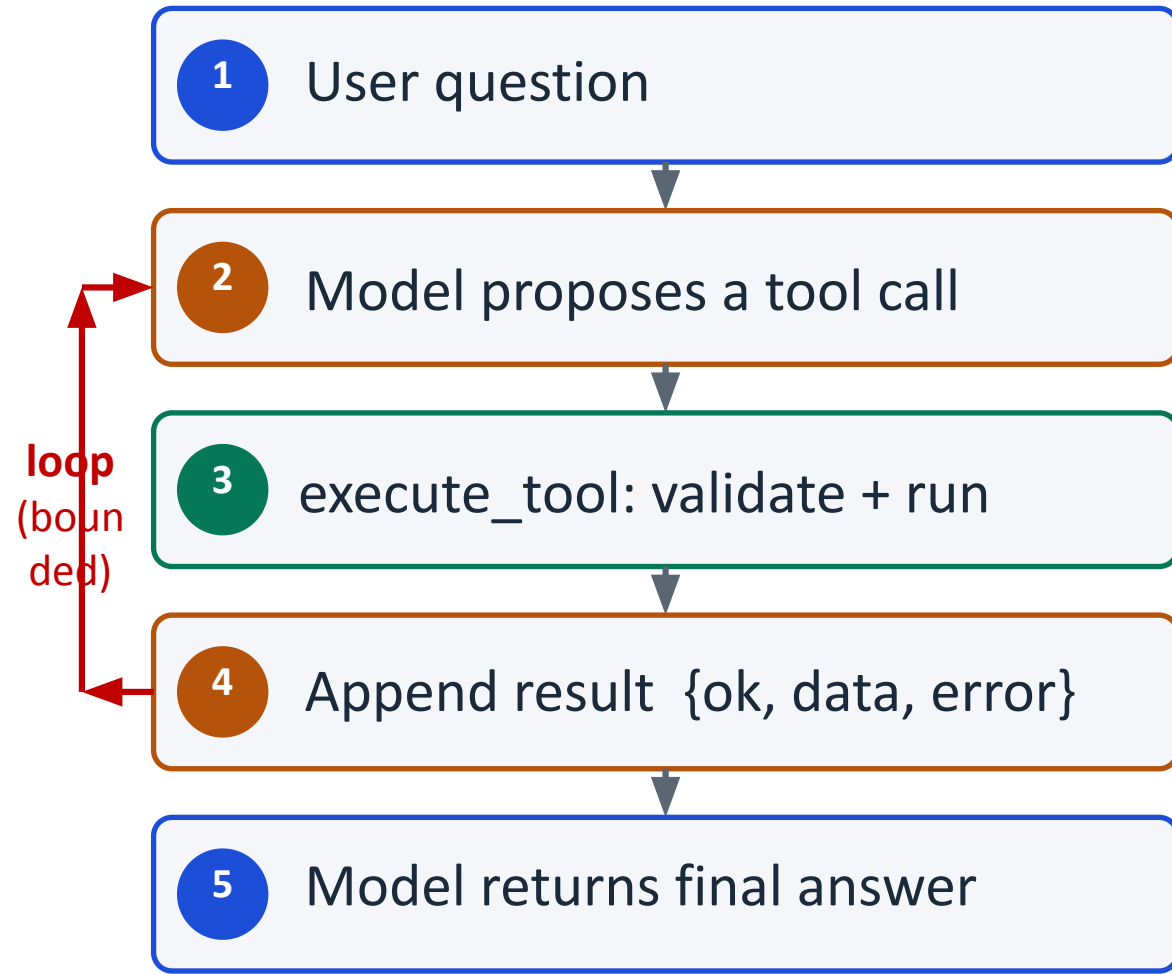
docs = SimpleDirectoryReader("data/notes").load_data()
nodes = SentenceSplitter(chunk_size=512,
chunk_overlap=80).get_nodes_from_documents(docs)

index = VectorStoreIndex(nodes)
query_engine = index.as_query_engine(similarity_top_k=6)

response = query_engine.query("Explain temperature vs top_p.")
print(response.response)
print(response.get_formatted_sources())
```

From a prompt to a tool-using agent

the model proposes a tool call; your code runs it and feeds the result back.



One safe entry point

```
def execute_tool(name, inputs):  
    validate(inputs, schema)    # reject early  
    data = run(name, inputs)  
    return json({"ok": True,  
                "data": data,  
                "error": None})
```

Why it stays safe

- Bound the loop: max steps + a stop reason
- Every call returns the same {ok,data,error}
- A rejection is data the model can retry on: The failure itself is useful information that helps the agent recover.

User: "Find the cheapest flight and tell me the weather at the destination."

Questions?